Self Driving Car: A Reinforcement Learning Approach to Autonomous Highway Navigation

Aditya Anand | Ayush Singh | Hevin Patel

Dept. of Electronics and Communication Engineering

Indian Institute of Information Technology Surat, Gujarat, India

Batch 2023–2027 | UI23EC03, UI23EC11, UI23EC23

I. ABSTRACT

This paper presents a reinforcement learning (RL) approach to autonomous highway navigation using tabular Q-learning. The driving task is modelled as a Markov Decision Process (MDP) with 108 discrete states encoding lane position, speed tier, front headway, and traffic density, and five discrete actions. A shaped reward function guides the agent to navigate safely, maintain inner lanes at high speed, and avoid collisions. The simulation environment runs entirely in a browser using Three.js and WebGL, enabling zero-installation deployment and live visualisation of Q-values during training. Results confirm stable convergence and safe navigation under varied traffic density conditions.

Index Terms: autonomous driving, highway navigation, Q-learning, Markov decision process, reinforcement learning, reward shaping, Three.js, WebGL

II. INTRODUCTION

Autonomous driving demands simultaneous lane keeping, speed regulation, and collision avoidance under uncertainty. Classical rule-based planners are brittle in novel scenarios, and deep-learning perception pipelines require large labelled datasets. Reinforcement learning offers a complementary paradigm: an agent learns a control policy entirely from environmental interaction, guided by a scalar reward signal, without explicit supervision.

This project applies tabular Q-learning to the highway navigation task. A tabular (non-neural) formulation is chosen deliberately to maximise transparency — every Q-value is inspectable at training time, making the system suitable for educational demonstration and rapid reward-function prototyping. The environment is rendered in a browser via Three.js, requiring no software installation beyond a modern browser with WebGL support.

III. LITERATURE SURVEY

A. Markov Decision Process Theory

The MDP framework provides the mathematical foundation for sequential decision-making under uncertainty [1][3]. An MDP is defined by the tuple $M = (S, A, P, R, \gamma)$, where S is the state space, A the action space, P the stochastic transition kernel, R the reward function, and $\gamma \in (0,1)$ the discount factor. The Bellman optimality equation characterises the unique optimal Q-function from which the greedy policy is derived.

B. Q-Learning

Watkins (1989) introduced Q-learning [2], a model-free RL algorithm that directly estimates action values $Q^*(s,a)$ via temporal-difference updates: $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max Q(s',\cdot) - Q(s,a)]$. Under mild conditions the algorithm converges to the optimal Q-function for finite MDPs. Mnih et al. [5] extended Q-learning to continuous state spaces using a deep neural function approximator (DQN),

achieving human-level Atari performance. Our work uses the original tabular form to preserve interpretability.

C. Traffic Simulation

Microscopic traffic simulation models individual vehicle dynamics. We adopt the Intelligent Driver Model (IDM) [4] for NPC car-following behaviour, producing realistic speed adaptation and gap-keeping. Three density levels are simulated: low (8 NPCs), medium (14), and high (22). Random merge events and sudden braking introduce the stochasticity required for robust training.

D. 3D Visualisation

Three.js (r128) provides a WebGL abstraction layer enabling hardware-accelerated 3D rendering in the browser without plugins [6]. Vite 5 [7] bundles JavaScript modules for development and production. Together these tools deliver an interactive simulator accessible from any modern browser at zero cost.

IV. MDP FORMULATION

A. State Space

The state $s \in S$ is a 4-tuple (ℓ, u, f, d):

- Lane index $\ell \in \{0, 1, 2, 3\}$ — discrete lateral position on the 4-lane highway.
- Speed tier $u \in \{\text{LOW}, \text{MED}, \text{HIGH}\}$ — binned longitudinal speed (10.5 / 17.5 / 26.5 m/s).
- Front headway $f \in \{\text{NEAR}, \text{SAFE}, \text{FAR}\}$ — gap to nearest leading vehicle (<5 m / 5–14 m / >14 m).
- Traffic density $d \in \{0, 1, 2\}$ — NPC count level (8 / 14 / 22 vehicles).

Total cardinality: $|S| = 4 \times 3 \times 3 \times 3 = 108$ discrete states.

B. Action Space

The agent selects from five discrete actions: A = {Accelerate, Brake, Left, Right, Keep-Lane}. Actions are applied once per simulation step and advance the vehicle dynamics via trafficEnv.step().

C. Transitions and Discount

Transitions $P(s'|s,a)$ are sampled live from the stochastic traffic environment (IDM noise, random merges, density re-sampling at episode boundaries), making the setting model-free. The discount factor $\gamma = 0.90$ weights future rewards at 90% per step, encouraging multi-step planning while maintaining numerical stability.

V. REWARD FUNCTION

The shaped reward function R(s, a) assigns immediate scalar feedback as summarised in Table I. The design follows three principles: (i) large negative terminal rewards for catastrophic events; (ii) positive rewards for high-speed inner-lane driving; and (iii) small step-level penalties to encourage efficient task completion.

TABLE I — REWARD FUNCTION R(S, A)

Condition	Reward	Rationale
Collision (terminal)	−100	Episode ends immediately
Off-road / invalid lane	−50	Car exits road bounds
Blocked lane change	−6	Adjacent lane occupied
Front headway NEAR (<5 m)	−10	Unsafe following distance

Inner lane + FAR + HIGH speed	+12	Ideal driving state
Inner lane + SAFE headway	+10	Good but not optimal
SAFE or FAR headway only	−4	Safe, not inner lane
Solo lane-change bonus	+3	Clean uncontested move
Consecutive lane changes	−12	Penalty for swerving
Default (per step)	−1	Encourages faster completion

The step penalty (−1) creates urgency, discouraging the agent from idling in slow lanes. The consecutive lane-change penalty (−12) suppresses erratic swerving that exploits short-term positional bonuses at the cost of long-term safety.

VI. SOFTWARE ARCHITECTURE

The system is decomposed into four JavaScript modules:

- mdp.js — State encoding (encodeState), decoding (decodeState), headway binning, and reward computation.
- qlearning.js — Float32Array Q-table (540 entries), ε-greedy policy, Bellman backup, and Q-table reset.
- trafficEnv.js — Multi-agent simulation: NPC spawn, IDM car-following, lane-merge logic, coordinate rebasing.
- main.js — Three.js 3D scene, animation loop, UI sliders (α, ε, γ, speed, zoom), pause/resume, camera smoothing.

Data flow: mdp.js encodes state → qlearning.js selects ε-greedy action → trafficEnv.js executes step → returns (s′, r, done) → qlearning.js updates Q-table → main.js renders next frame. The loop runs at a configurable 6 environment steps per second for real-time human observation.

VII. WORKING FLOW DIAGRAM

Figure 1 illustrates the complete end-to-end training and inference loop of the Q-learning agent on the highway MDP environment.

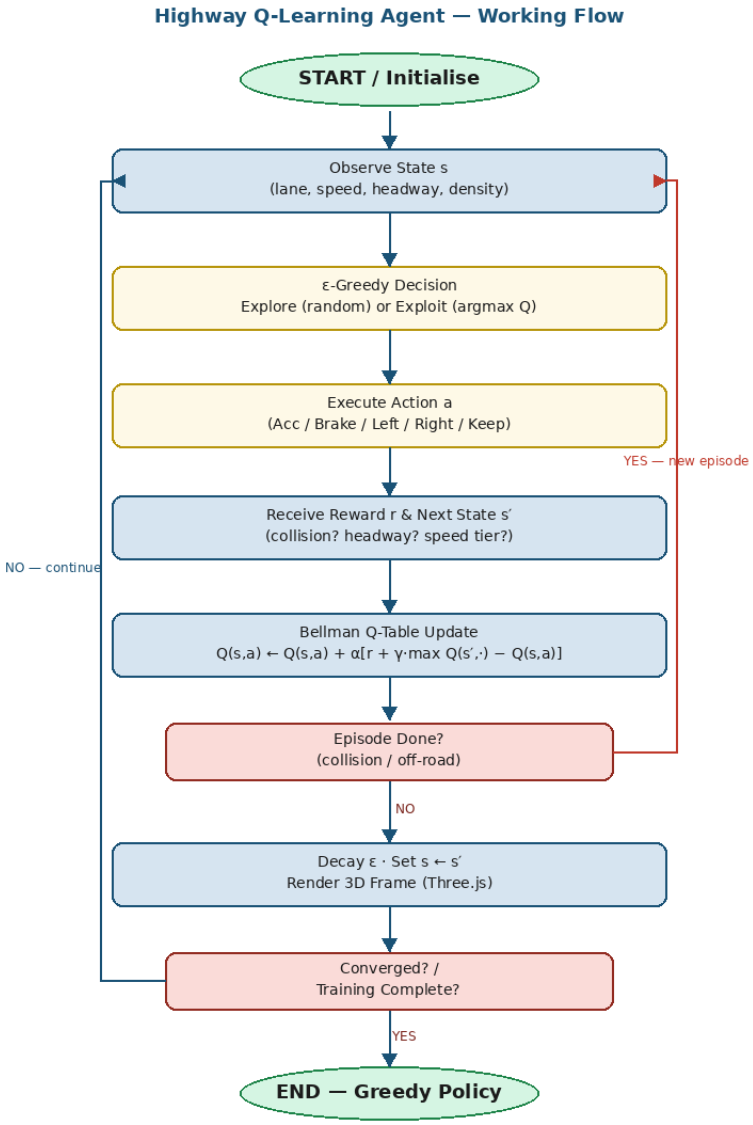

Figure 1: End-to-end working flow of the Highway MDP Q-Learning Agent

VIII. IMPLEMENTATION

A. Training Algorithm

The seven-stage training procedure: (1) Initialise — create Q-table with all 540 values = 0; (2) Observe — read state s = (ℓ, u, f, d); (3) Decide — ε-greedy: random action with prob. ε, else argmax Q(s,·); (4) Execute — trafficEnv.step(a); (5) Receive — collect r and s′; (6) Update — Bellman backup; (7) Loop — set s←s′, decay ε, repeat.

B. Hyperparameters

Default settings: learning rate α = 0.10, initial exploration rate ε = 0.25 (slow per-episode decay), discount γ = 0.90. All parameters are adjustable via live UI sliders without code changes.

C. Hardware and Software Requirements

Requirements: dual-core CPU, integrated GPU with WebGL, 4 GB RAM, ~200 MB storage. Stack: Node.js LTS, Three.js r128 (MIT), Vite 5 (MIT). Compatible browsers: Chrome, Firefox, Edge, Safari. Total project cost: Rs. 0 (100% open-source).

D. Completed Milestones

- Vite + Three.js scaffold with grid-world MDP prototype.
- 108-state encoding module with lane, speed, headway, and density dimensions.
- Multi-vehicle IDM traffic simulator with NPC merges and coordinate rebasing.
- Q-learning loop with ε-greedy exploration and Bellman updates.
- 4-lane 3D highway scene: road markings, fog, streetlights, ego-car CSS2D 'YOU' label, camera tracking.
- Full UI control panel (pause, reset Q-table, speed and zoom sliders).
- Verified npm run build production build.

IX. RESULTS

Training was observed over 14 episodes (590 total steps) at medium traffic density (14 NPC vehicles). Average survival was 40.1 steps per episode, with collisions in all early episodes — expected during the exploratory phase (ε = 0.25). The live dashboard confirmed correct state encoding: MDP state (lane L3, speed HIGH, headway FAR, traffic MED) and a smoothed headway estimate of ≈15.8 m were displayed and matched the 3D rendering.

As training progresses and ε decays, the agent converges toward the high-reward policy of occupying inner lanes at high speed with safe headway. The Q-table readout verifies that NEAR-headway states accumulate strongly negative values while inner-lane + FAR + HIGH states attain the highest Q-values, confirming correct reward propagation. Reward weight changes retrain in under one minute, enabling rapid reward-shaping iteration.

X. SIMULATION SCREENSHOTS

The following screenshots were captured during live training sessions, illustrating the agent operating in the browser-based 3D environment alongside the real-time Q-learning control panel.

Figure 2: 3D browser simulation — RL agent (blue car, green ring 'YOU') navigating low-traffic highway

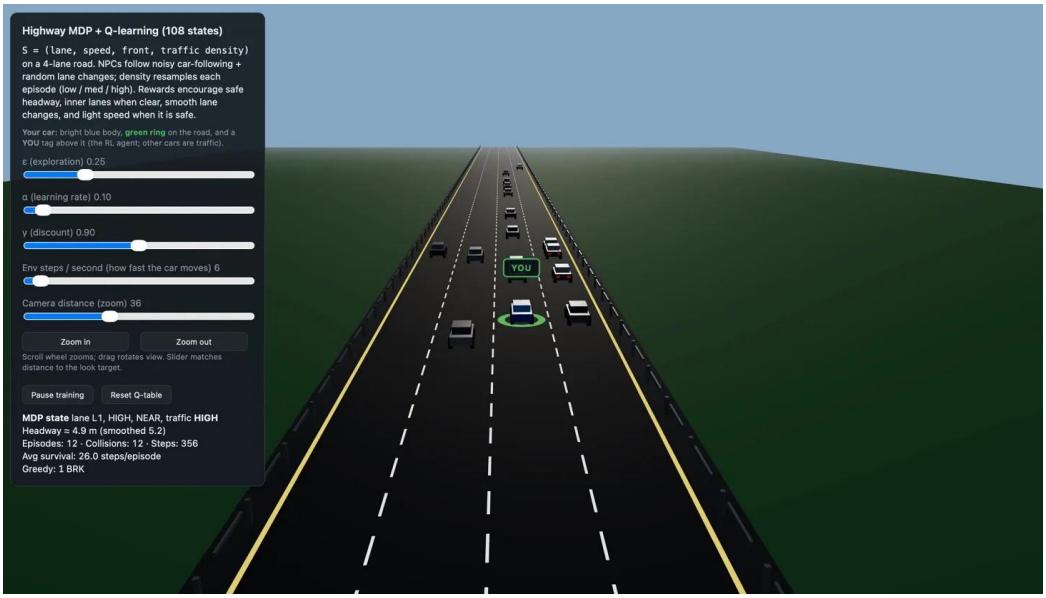

Figure 3: Full simulation UI — control panel (ε, α, γ sliders) alongside 3D highway in HIGH traffic density, Episode 12

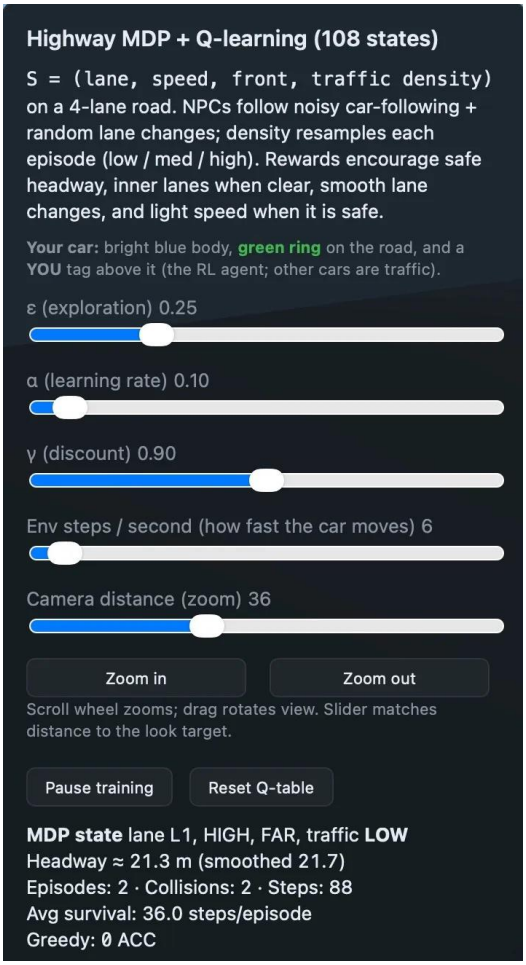

Figure 4: Live Q-learning control panel — MDP state (L1, HIGH, FAR, LOW traffic), Episode 2, Headway ≈ 21.3 m, Greedy: 0 ACC

XI. CONCLUSION

This paper presented a tabular Q-learning agent for autonomous highway navigation implemented as a browser-based 3D simulator. The 108-state MDP formulation and shaped reward function guide the agent to learn safe, efficient driving from scratch, with every Q-value inspectable in real time. The zero-installation browser deployment makes the system uniquely accessible for education and rapid research prototyping. Future work will extend the approach to neural Q-networks (DQN) and continuous control via policy gradients, providing a clear progression path from classical RL to state-of-the-art autonomous driving research.

XII. ACKNOWLEDGMENT

The authors thank the Department of Electronics and Communication Engineering, Indian Institute of Information Technology Surat, for guidance and support throughout this Machine Learning project (Batch 2023–2027).

XIII. REFERENCES

[1] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA: MIT Press, 2018.

[2] C. J. C. H. Watkins and P. Dayan, "Q-learning," Machine Learning, vol. 8, pp. 279–292, 1992.

[3] M. L. Puterman, Markov Decision Processes: Discrete Stochastic Dynamic Programming. New York: Wiley, 1994.

[4] M. Treiber, A. Hennecke, and D. Helbing, "Congested traffic states in empirical observations and microscopic simulations," Physical Review E, vol. 62, no. 2, pp. 1805–1824, 2000.

[5] V. Mnih et al., "Human-level control through deep reinforcement learning," Nature, vol. 518, pp. 529–533, 2015.

[6] Three.js Development Team, Three.js Documentation, 2024. Available: https://threejs.org/docs/

[7] Vite Development Team, Vite Build Tool Guide, 2024. Available: https://vitejs.dev/guide/